

# **Core OOP Concepts in Java**

*Polymorphism, Abstraction Mechanisms,  
and Design Principles*

# Learning Objectives

By the end of this session, students will be able to:

- **Differentiate** between Compile-time and Runtime Polymorphism in Java.
- **Implement** Polymorphic behavior using method overriding and dynamic binding.
- **Choose** appropriately between Abstract Classes and Interfaces for design contracts.
- **Apply** Java 8+ interface features (default/static methods).
- **Evaluate** class designs using SOLID principles, cohesion, and coupling metrics.
- **Refactor** rigid Java code into flexible, maintainable structures.

# Polymorphism Overview

**Definition:** "Many forms." The ability of an object to take on many forms. In Java, this allows a superclass reference to refer to a subclass object.

## Two Types in Java:

- **Compile-time Polymorphism (Static Binding)**  
→ Method Overloading.
- **Runtime Polymorphism (Dynamic Binding)**  
→ Method Overriding.

## Why it Matters:

- Enables writing generic code that works with objects of multiple types.
- Core foundation for Design Patterns (Strategy, Factory, Command).

# Compile-time Polymorphism (Overloading)

## Definition:

Multiple methods in the same class with the same name but different parameters (signature).

## Resolution:

Decided by the **Compiler** based on reference type and arguments.

# Compile-time Polymorphism (Overloading)

## Java Example

```
public class Calculator {  
    public int add(int a, int b) {  
        return a + b;  
    }  
    //(Overloaded)  
    public double add(double a, double b) {  
        return a + b;  
    }  
    //Overloaded - different order  
    public int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

# Runtime Polymorphism (Overriding)

## Definition:

A subclass provides a specific implementation of a method already defined in its superclass.

## Resolution:

Decided by the **JVM** at runtime based on the actual **object type**, not the reference type.

# Runtime Polymorphism (Overriding)

## Java Example

```
class Animal {  
    void sound() {  
        System.out.println("Some sound");  
    }  
}
```

```
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Bark");  
    }  
}
```

---

```
Animal myPet = new Dog(); // Upcasting  
myPet.sound(); // Output: "Bark" (Dynamic Method Dispatch)
```

# Polymorphism in Collections

## Real-world Application:

Java Collections rely heavily on polymorphism to store heterogeneous objects.

## Example:

```
// objects can be Circle, Square, etc.
List<Shape> shapes = new ArrayList<>();
shapes.add(new Circle());
shapes.add(new Square());
for (Shape s : shapes) {
    // Polymorphic call: draws shape based on actual object
    s.draw();
}
```

## Benefit:

- Code is open for extension (add [Triangle](#) later) without modification.
- Eliminates excessive [if-else](#) or [switch](#) statements based on type.

# Abstract Classes in Java

## **Definition:**

A class declared with the abstract keyword. It cannot be instantiated directly.

## **Purpose:**

- To provide a common template for subclasses.
- To enforce certain methods must be implemented by subclasses.

## **Key Features:**

- Can have both abstract (no body) and concrete (with body) methods.
- Can have state (instance variables).
- Can have constructors (called by subclasses via super).

# Abstract Classes in Java

```
public abstract class Employee {  
    private String name;  
    public Employee(String name) {  
        this.name = name;  
    }  
    // Concrete method  
    public void work() {  
        System.out.println("Working...");  
    }  
    // Abstract method (must be implemented by subclass)  
    public abstract double calculateSalary();  
}
```

# Interfaces in Java

## Definition:

A reference type that can contain only constants, method signatures, default methods, static methods, and nested types.

## Purpose:

- To define a contract or capability (e.g., Comparable, Runnable).
- To support multiple inheritance of type.

## Java 8+ Enhancements:

- **default methods:** Provide implementation within the interface.
- **static methods:** Utility methods related to the interface.

# Interfaces in Java

## Java Example:

```
public interface Flyable {  
    void fly(); // Abstract  
  
    default void land() { // Default method  
        System.out.println("Landing...");  
    }  
  
    static boolean isAirborne() { // Static method  
        return true;  
    }  
}
```

# Abstract Class vs. Interface (Decision Matrix)

| Feature          | Abstract Class                     | Interface                            |
|------------------|------------------------------------|--------------------------------------|
| Relationship     | "Is-A" (Inheritance)               | "Can-Do" (Capability)                |
| Inheritance      | Single (`extends`)                 | Multiple (`implements`)              |
| State            | Can have instance variables        | Only `public static final` constants |
| Constructors     | Yes                                | No                                   |
| Method Types     | Abstract + Concrete                | Abstract + Default + Static          |
| Access Modifiers | Any (`private`, `protected`, etc.) | Mostly `public` (methods)            |

## Design Guideline:

- Use **Abstract Class** if you need to share code or state among closely related classes.
- Use **Interface** if you expect unrelated classes to implement your contract, or need multiple inheritance of type.

# Sound Class Design Principles (SOLID)

## Overview of SOLID in Java:

1. S - Single Responsibility Principle (SRP)
  - A class should have one reason to change.
2. O - Open/Closed Principle (OCP)
  - Open for extension, closed for modification.
3. L - Liskov Substitution Principle (LSP)
  - Subtypes must be substitutable for base types.
4. I - Interface Segregation Principle (ISP)
  - Many specific interfaces are better than one general one.
5. D - Dependency Inversion Principle (DIP)
  - Depend on abstractions, not concretions.

# Cohesion and Coupling

## Cohesion (Internal Strength):

- **High Cohesion:** Class methods and fields are strongly related.
- **Low Cohesion:** Class does unrelated tasks (God Class).
- **Goal:** Maximize Cohesion.

## Coupling (External Dependency):

- **Low Coupling:** Classes are independent; changes in one don't affect others.
- **High Coupling:** Classes are tightly intertwined.
- **Goal:** Minimize Coupling.

# Cohesion and Coupling

## Java Example (Reducing Coupling):

```
// ✗ High Coupling
class OrderService {
    private MySQLDatabase db = new MySQLDatabase();
}

// ✓ Low Coupling (DIP)
class OrderService {
    private Database db; // Depends on abstraction
    public OrderService(Database db) { // Injection
        this.db = db;
    }
}
```

# Composition Over Inheritance

## **The Problem with Inheritance:**

- Breaks encapsulation (parent changes break child).
- Rigid hierarchy (cannot change at runtime).
- Java limits you to single class inheritance.

## **The Composition Solution:**

- "Has-A" relationship.
- Delegate behavior to helper objects.
- More flexible and testable.

# Composition Over Inheritance

## Java Example:

```
// X Inheritance Abuse
// class Stack extends Vector { ... }

// ✓ Composition
class Stack {
    private List<Object> elements = new ArrayList<>();

    public void push(Object o) { elements.add(o); }
    public Object pop() { ... }
}
```

## Rule of Thumb:

Favor composition when code reuse is needed without an "Is-A" relationship.

# Integrated Case Study: Payment System

## Refactoring to Sound Design

### 1. Interface (Contract):

```
public interface PaymentProvider {  
    void pay(double amount);  
}
```

### 2. Concrete Implementations (Polymorphism):

```
public class PayPal implements PaymentProvider { ... }  
public class CreditCard implements PaymentProvider { ... }
```

### 3. Concrete Implementations (Polymorphism):

```
public class Checkout {  
    private PaymentProvider provider; // Abstraction  
    public Checkout(PaymentProvider provider) {  
        this.provider = provider; // Injected dependency  
    }  
    public void completeOrder(double amount) {  
        provider.pay(amount); // Polymorphic call  
    }  
}
```

# Common Anti-Patterns to Avoid

## ⊗ **The God Object:**

A class that controls too much of the system (violates SRP).

*Fix:* Extract responsibilities into separate classes.

## ⊗ **Flag Arguments:**

```
public void create(boolean isMember)
```

*Fix:* Split into createMember() and createGuest().

## ⊗ **Primitive Obsession:**

Using String for phone numbers or emails everywhere.

*Fix:* Create value objects (class PhoneNumber, class Email).

## ⊗ **Switch Statements on Types:**

```
if (obj instanceof Dog) ... else if (obj instanceof Cat)
```

*Fix:* Use polymorphism (obj.speak()).

# Summary & Key Takeaways

- ◆ **Polymorphism** allows flexible code via Overloading (compile-time) and Overriding (runtime).
- ◆ **Abstract Classes** share code/state; **Interfaces** define contracts/capabilities.
- ◆ **SOLID Principles** guide maintainable Java architecture
- ◆ **Low Coupling & High Cohesion** are metrics of good design.
- ◆ **Composition** is often safer and more flexible than Inheritance.
- ◆ **Program to Interfaces:** Always declare variables using the most abstract type possible (List vs ArrayList).

# Practice Exercises

- **Polymorphism Challenge:**  
Create a Shape hierarchy. Write a method `printArea(Shape s)` that works for any subclass without instanceof.
- **Interface Refactoring:**  
Identify a class with multiple responsibilities. Extract an interface and implement Dependency Injection.
- **Abstract vs. Interface:**  
Design a Vehicle system. Decide which parts go into an Abstract Class (e.g., `engineStatus`) vs. Interface (e.g., `Flyable`).
- **SOLID Audit:**  
Take a previous project assignment. Identify one violation of the Single Responsibility Principle and refactor it.

# Further Reading & Resources

## Books:

- *Effective Java* — Joshua Bloch (Items 1-10 cover OOP basics)
- *Head First Design Patterns* — Freeman & Freeman
- *Clean Architecture* — Robert C. Martin

## Online:

- [Oracle Java Interfaces Tutorial](#)
- [Refactoring.guru — SOLID Principles](#)
- [Java Design Patterns \(GitHub\)](#)

# Q&A + Discussion

- 🔗 **Question:** Can an interface have a constructor?
- 🔗 **Question:** What happens if a class implements two interfaces with the same default method?
- 🔗 **Question:** How does Polymorphism impact performance in Java (dynamic dispatch)?
- 🔗 **Question:** When is it okay to break the Liskov Substitution Principle?
- 💬 **Discussion:** Share an example of a "God Class" you encountered in legacy code. How would you break it down?